0 1 2

XÂU

manacher("racecar")



đọc xuôi = đọc ngược

VÀ THUẬT TOÁN

SO KHỚP · BĂM · HẬU TỐ

KMP · Hàm Z · Băm đa thức · Manacher · Trie · Aho–Corasick · Mảng hậu tố · Ôtômát hậu tố

π[i] = |biên dài nhất của s[0..i]|

tiền tố trùng hậu tố, không tính chính nó

h(s[l..r]) trong O(1)

băm tiền tố, so sánh hai xâu con

ô cam: khác nhau → trượt mẫu, không lùi lại trên s

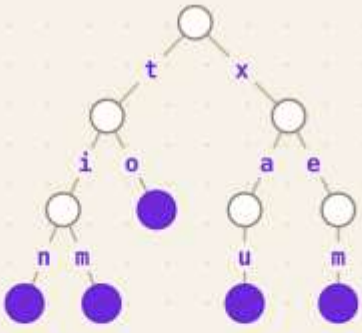




mảng hậu tố của "banana"

sa	lcp	
5	–	a
3	1	a n a
1	3	a n a n a
0	0	b a n a n a
4	0	n a
2	2	n a n a

trie: tin · tim · to · xau · xem



tìm t trong s: O(n·m) → O(n + m)

01	02	03	04	05	06	07	08	09
Xâu cơ bản	Duyệt xâu	So khớp mẫu	Băm xâu	Xâu đối xứng	QHĐ trên xâu	Trie · Aho–C.	Mảng hậu tố	Nâng cao

TẬP 1 · LÝ THUYẾT — 24 bài học · 9 chương · lý thuyết đầy đủ trước, bài toán sau

Chuỗi · Mẫu · Hậu tố · Đối xứng · Quy hoạch động

Tổng hợp và biên soạn: Lê Thành Nam

Lê Thành Nam

LỜI NÓI ĐẦU

Xâu kí tự có mặt trong hầu hết các đề thi học sinh giỏi Tin học, từ cấp tỉnh tới quốc gia và quốc tế. Có khi đó là một bài "dễ" về chuẩn hoá văn bản hay cộng hai số rất lớn; có khi là bài tìm một mẫu trong văn bản dài hàng triệu kí tự, đếm số xâu con khác nhau, tìm xâu con đối xứng dài nhất, hay đếm số xâu không chứa một từ cấm. Điều đặc biệt của chủ đề này là khoảng cách rất xa giữa cách làm tự nhiên và cách làm đúng: với xâu dài 10^6, cách so từng vị trí có thể cần tới 10^{12} phép tính, trong khi một thuật toán tuyến tính chỉ cần vài triệu.

Cuốn sách này trình bày một cách có hệ thống toàn bộ những kiến thức về xâu mà một học sinh giỏi cần biết, theo đúng trình tự từ dễ tới khó:

• Nền tảng: kí tự và bảng mã, kiểu string và độ phức tạp của từng thao tác, duyệt và biến đổi xâu, số lớn (Chương 1); các kĩ thuật duyệt khéo léo: mảng đếm và mặt nạ chẵn lẻ, hai con trỏ, thứ tự từ điển và tham lam, ngăn xếp và dãy ngoặc (Chương 2).

• So khớp mẫu: hàm tiền tố và thuật toán KMP, các ứng dụng của hàm tiền tố, hàm Z (Chương 3); băm xâu và chặt nhị phân (Chương 4).

• Xâu đối xứng: quy hoạch động và mở rộng từ tâm, thuật toán Manacher, cây đối xứng (Chương 5).

• Quy hoạch động trên xâu: dãy con chung dài nhất, khoảng cách sửa, quy hoạch động đếm, quy hoạch động trên ôtômát (Chương 6).

• Cấu trúc dữ liệu cho xâu: trie và Aho – Corasick (Chương 7); mảng hậu tố và mảng LCP (Chương 8); ôtômát hậu tố, xâu xoay vòng nhỏ nhất và phân tích Lyndon (Chương 9).

Bộ tài liệu gồm ba tập đi cùng nhau:

• Tập 1: Lý thuyết (cuốn sách này) gồm 24 bài học chia thành 9 chương. Mỗi bài học trình bày đầy đủ lý thuyết trước (định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình vẽ), rồi mới vào một bài toán lập trình với cách chậm, cách nhanh và số đo thời gian thật.

• Tập 2: Bài tập gồm 72 bài: mỗi bài học có ba bài tập ở ba mức độ, đề viết theo đúng thể thức đề thi học sinh giỏi, ràng buộc chia theo subtask.

• Tập 3: Hướng dẫn giải phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm, kể cả những cách sai hoặc đúng nhưng chậm.

Sách được viết cho người mới bắt đầu: chỉ cần biết C++ cơ bản (biến, vòng lặp, mảng, hàm) và sắp xếp. Mọi con số trong sách (thời gian chạy, kết quả chạy tay, đáp số các câu hỏi tự kiểm tra) đều là số đo và kết quả thật. Mọi chương trình đều đã được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Một điều cần nhớ khi học chủ đề này: chương trình xử lí xâu sai thường vẫn qua được các test ngẫu nhiên. Văn bản ngẫu nhiên hiếm khi có cấu trúc lặp, nên cách so từng vị trí chạy nhanh như KMP; băm modulo 2^{64} không bao giờ đụng độ trên dữ liệu ngẫu nhiên; mở rộng từ tâm chạy như O(n). Chỉ những test được thiết kế riêng (xâu toàn một kí tự, xâu Thue – Morse, mẫu dạng

LÊ Thành Nam

"aa...ab") mới làm lộ ra chúng. Vì vậy sách luôn giải thích vì sao một thuật toán đúng và vì sao nó nhanh, và luôn đo thời gian trên những test xấu nhất.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học gồm hai phần lớn:

Phần	Nội dung
Mục tiêu, Khởi động	Những gì em sẽ làm được sau bài học, và một tình huống dẫn tới ý tưởng chính.
PHẦN LÝ THUYẾT	Định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình minh họa. Đọc phần này với giấy bút, tự tính lại các mảng π, z, d, sa, lcp trong ví dụ.
PHẦN BÀI TOÁN	Một bài toán lập trình dùng đúng lý thuyết vừa học: cách chậm (kèm số đo thời gian thật), cách nhanh, chương trình hoàn chỉnh, phân tích độ phức tạp.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA	Khái niệm mới, cần hiểu chính xác.
ĐỊNH LÍ, TÍNH CHẤT	Kết quả quan trọng, thường kèm chứng minh.
CHỨNG MINH	Lập luận vì sao định lí đúng.
VÍ DỤ, CHẠY TAY	Chạy tay trên số liệu cụ thể.
CÔNG THỨC	Công thức cần nhớ.
Ý TƯỞNG, THUẬT TOÁN	Hướng giải hoặc các bước của một phương pháp.
BÀI TOÁN	Đề bài lập trình của bài học.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và số đo thời gian thật.
VÌ SAO ĐÚNG?	Giải thích thêm một bước khó.
LƯU Ý	Chi tiết dễ bỏ sót.
MỞ RỘNG	Kiến thức nâng cao, có thể bỏ qua khi đọc lần đầu.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Kí hiệu. Xâu s có độ dài n = |s|, các kí tự đánh số từ 0: s[0], s[1], ..., s[n − 1], đúng như trong C++. Kí hiệu s[l..r] là đoạn gồm các kí tự từ vị trí l tới vị trí r (cả hai đầu), độ dài r − l + 1; s[i..] là hậu tố bắt đầu tại i. Phép ghép hai xâu viết là a + b. Thứ tự giữa hai xâu luôn là thứ tự từ điển (Bài 6). Riêng đề bài (dữ liệu vào, kết quả) thường đánh số vị trí từ 1, theo thói quen của các đề thi; khi đó sách ghi rõ. log n luôn hiểu là $\log_2 n$. Trong hình vẽ, ô cam thường là đối tượng đang xét, ô xanh lá là kết quả, ô vàng là phần cần chú ý, ô xám là phần bị bỏ qua.

Thời gian chạy. Các số đo trong sách là thời gian CPU trên một máy tính cá nhân thông thường, biên dịch với g++ -O2 -std=c++14. Máy chấm của em có thể nhanh hơn hoặc chậm hơn vài lần,

Lê Thanh Nam

nhưng tỉ lệ giữa các cách làm thì không đổi: một thuật toán O(n·m) chậm hơn thuật toán O(n + m) hàng nghìn lần ở bất kì máy nào.

Lời khuyên khi học. Với mỗi thuật toán, hãy tự chạy tay trên một xâu độ dài 6 tới 10 trước khi đọc chương trình. Với mỗi chương trình, hãy tự hỏi: biến này có ý nghĩa gì sau mỗi vòng lặp (bất biến)? Con trỏ nào chỉ tăng, và vì thế tổng số bước là bao nhiêu? Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **4**

CHƯƠNG 1. XÂU VÀ CÁC THAO TÁC CƠ BẢN **8**

Bài 1. Kí tự, bảng mã và kiểu string 8
Bài 2. Duyệt và biến đổi xâu 18
Bài 3. Số lớn biểu diễn bằng xâu 26

CHƯƠNG 2. KĨ THUẬT DUYỆT TRÊN XÂU **34**

Bài 4. Mảng đếm và mảng cộng dồn trên xâu 34
Bài 5. Hai con trỏ và cửa sổ trượt trên xâu 42
Bài 6. Thứ tự từ điển và tham lam trên xâu 49
Bài 7. Ngăn xếp và dãy ngoặc 57

CHƯƠNG 3. SO KHỚP MẪU **64**

Bài 8. So khớp mẫu và thuật toán KMP 64
Bài 9. Ứng dụng của hàm tiền tố 74
Bài 10. Hàm Z 81

CHƯƠNG 4. BĂM XÂU **88**

Bài 11. Băm xâu 88
Bài 12. Băm kết hợp chặt nhị phân 97

CHƯƠNG 5. XÂU ĐỐI XỨNG **103**

Bài 13. Xâu đối xứng: quy hoạch động và mở rộng từ tâm 103
Bài 14. Thuật toán Manacher 110
Bài 15. Cây đối xứng 117

CHƯƠNG 6. QUY HOẠCH ĐỘNG TRÊN XÂU **124**

Bài 16. Dãy con chung dài nhất và khoảng cách sửa 124
Bài 17. Quy hoạch động đếm trên xâu 132
Bài 18. Quy hoạch động trên ôtômát 138

CHƯƠNG 7. NHIỀU MẪU: TRIE VÀ AHO – CORASICK **144**

Bài 19. Cây tiền tố (Trie) 144
Bài 20. Thuật toán Aho – Corasick 151

CHƯƠNG 8. MẢNG HẬU TỐ **158**

Bài 21. Mảng hậu tố 158
Bài 22. Mảng LCP và ứng dụng 166

CHƯƠNG 9. CẤU TRÚC XÂU NÂNG CAO **173**

Lê Thành Nam

Bài 23. Ôtômát hậu tố 173
Bài 24. Xâu xoay vòng nhỏ nhất và phân tích Lyndon 182

PHỤ LỤC: BẢNG TRA CỨU NHANH 188

Lê Thành Nam

CHƯƠNG 1

XÂU VÀ CÁC THAO TÁC CƠ BẢN

Xâu là gì và mỗi thao tác trên xâu tốn bao nhiêu. Nhiều lỗi "chương trình đúng nhưng quá chậm" của học sinh bắt nguồn từ chính những điều tưởng như đơn giản ở chương này.

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Kí tự, bảng mã và kiểu string
- **Bài 2.** Duyệt và biến đổi xâu
- **Bài 3.** Số lớn biểu diễn bằng xâu

BÀI 1 Kí tự, bảng mã và kiểu string

Học xong bài này, em sẽ:

✓ Hiểu vì sao mỗi kí tự là một con số và dùng thành thạo các phép biến đổi c − '0', c − 'a', đổi hoa thành thường.

✓ Phân biệt chính xác xâu con, tiền tố, hậu tố, dãy con; đếm được số lượng của chúng.

✓ Đọc xâu đúng cách: `cin >> s`, `getline`, và tránh lỗi "dòng rỗng" khi trộn hai cách đọc.

✓ Biết độ phức tạp của từng thao tác trên `string`, đặc biệt là cái bẫy `s = s + c`.

■ KHỞI ĐỘNG

Một chương trình đọc một dòng gồm 10^6 kí tự, giữ lại các chữ cái và chữ số. Bạn An viết `res = res + c;`, bạn Bình viết `res += c;`. Hai chương trình chỉ khác nhau đúng **một dấu cộng**. Chương trình của Bình chạy 0,012 giây. Chương trình của An chạy **10,8 giây**, chậm hơn gần một nghìn lần. Bài học này giải thích vì sao, và trang bị cho em những hiểu biết nền tảng nhất về xâu mà mọi bài học sau đều dựa vào.

PHẦN LÝ THUYẾT

1. Kí tự là một con số

Máy tính chỉ lưu được các con số. Để lưu chữ, người ta quy ước mỗi kí tự ứng với một số nguyên gọi là **mã** của nó. Bảng quy ước phổ biến nhất là **bảng mã ASCII**, gồm 128 kí tự đánh số từ 0 tới 127.

■ KIỂU CHAR VÀ BẢNG MÃ ASCII

Trong C++, kiểu char chiếm 1 byte và lưu **mã** của một kí tự. Viết char c = 'A'; thì c thực chất chứa số 65. Ta có thể in (int)c để thấy mã, làm phép cộng trừ trên c như với số nguyên, và so sánh hai kí tự bằng cách so sánh hai mã.

Em không cần thuộc cả bảng ASCII. Chỉ cần nhớ ba khối quan trọng, mỗi khối là **một dãy mã liên tiếp**:

Khối	Mã	Ghi chú
'0', '1', ..., '9'	48 – 57	chữ số '0' có mã 48, không phải 0
'A', 'B', ..., 'Z'	65 – 90	26 chữ hoa liên tiếp
'a', 'b', ..., 'z'	97 – 122	26 chữ thường liên tiếp
dấu cách ' '	32	kí tự in được nhỏ nhất
xuống dòng ' '	10	kí tự điều khiển, không in ra



Hình 1.1. Ba khối mã liên tiếp: chữ số, chữ hoa, chữ thường

Vì các khối là liên tiếp, mọi phép biến đổi quen thuộc chỉ là một phép trừ hoặc cộng:

■ CÔNG THỨC

- Giá trị của chữ số c: **c − '0'**. Ví dụ '7' − '0' = 55 − 48 = 7.
- Thứ tự (tính từ 0) của chữ thường c trong bảng chữ cái: **c − 'a'**. Ví dụ 'd' − 'a' = 3.
- Đổi chữ hoa c thành chữ thường: **c − 'A' + 'a'** (tức là c + 32). Ngược lại: **c − 'a' + 'A'**.
- Kiểm tra c là chữ thường: **'a' ≤ c ≤ 'z'**; là chữ số: **'0' ≤ c ≤ '9'**.
- Kí tự ứng với số k (0 ≤ k ≤ 25): **'a' + k**; chữ số ứng với d (0 ≤ d ≤ 9): **'0' + d**.

Lê Thành Nam

■ VÍ DỤ

Đếm tần suất các chữ thường trong s = "banana". Dùng mảng cnt[0..25] toàn 0, với mỗi kí tự c tăng cnt[c − 'a']: 'b' tăng cnt[1], ba chữ 'a' tăng cnt[0] ba lần, hai chữ 'n' tăng cnt[13] hai lần. Kết quả cnt[0] = 3, cnt[1] = 1, cnt[13] = 2. In ra bằng vòng lặp k từ 0 tới 25, kí tự là (char)('a' + k).

Thư viện <cctype> có sẵn các hàm isdigit, isalpha, isupper, islower, isalnum, tolower, toupper.

■ LƯU Ý

• tolower(c) và toupper(c) trả về kiểu int. Viết cout << toupper(c); sẽ in ra **một con số** (ví dụ 65) chứ không phải chữ 'A'. Cần ép kiểu: cout << (char)toupper(c);, hoặc gán vào một biến char.

• Các hàm isalpha, isdigit, ... chỉ an toàn với kí tự ASCII. Với char mang giá trị âm (byte của chữ tiếng Việt), kết quả không xác định.

• **Chữ tiếng Việt có dấu không nằm trong bảng ASCII.** Theo chuẩn UTF-8, chữ "ê" chiếm 2 byte, chữ "ệ" chiếm 3 byte. Vì vậy string s = "Việt"; có s.size() bằng 6 chứ không phải 4. Đề thi học sinh giỏi gần như luôn chỉ dùng kí tự ASCII; nếu đề có tiếng Việt, hãy đọc kĩ cách đề định nghĩa "kí tự".

2. Xâu và các khái niệm cơ bản

■ XÂU

Một **xâu** (string) độ dài n là một dãy n kí tự s[0], s[1], ..., s[n − 1]. Xâu độ dài 0 gọi là **xâu rỗng**, kí hiệu "". Trong sách này chỉ số của xâu **luôn đánh từ 0** (như trong C++), và kí hiệu **s[l..r]** là đoạn gồm các kí tự s[l], s[l + 1], ..., s[r] (độ dài r − l + 1).

Bốn khái niệm sau xuất hiện trong mọi bài học của cuốn sách. Hãy phân biệt thật chính xác:

■ XÂU CON, TIỀN TỐ, HẬU TỐ, DÃY CON

• **Xâu con** (substring) của s là một đoạn **liên tiếp** s[l..r] với $0 \le l \le r \le n - 1$.

• **Tiền tố** (prefix) là xâu con bắt đầu từ đầu xâu: s[0..r]. **Hậu tố** (suffix) là xâu con kết thúc ở cuối xâu: s[l..n − 1].

• **Dãy con** (subsequence) nhận được khi **xoá đi một số kí tự** (có thể không xoá kí tự nào) và giữ nguyên thứ tự các kí tự còn lại. Các kí tự của dãy con **không cần liên tiếp** trong s.

• Tiền tố (hậu tố) khác chính s gọi là tiền tố (hậu tố) **thật sự**.



Hình 1.2. Xâu con, tiền tố, hậu tố và dãy con của s = "abracadabra"

Lê Thành Nam

Mọi xâu con đều là dãy con, nhưng điều ngược lại sai: "acd" là dãy con của "abracadabra" (lấy các kí tự ở vị trí 0, 4, 6) nhưng không là xâu con, vì ba kí tự ấy không đứng liền nhau.

■ ĐẾM THEO VỊ TRÍ

Với xâu s độ dài n:

- có đúng **n(n + 1)/2** xâu con khác rỗng (tính theo cặp vị trí (l, r));
- có đúng n tiền tố khác rỗng và n hậu tố khác rỗng;
- có đúng $\mathbf{2^n - 1}$ dãy con khác rỗng (tính theo tập vị trí được giữ lại).

■ CHỨNG MINH

Một xâu con khác rỗng ứng với một cặp (l, r), $0 \le l \le r \le n - 1$. Với mỗi l có n − l cách chọn r, nên tổng là $n + (n - 1) + ... + 1 = n(n + 1)/2$. Một tiền tố ứng với một giá trị r, một hậu tố ứng với một giá trị l. Một dãy con ứng với một tập con các vị trí được giữ lại; có 2^n tập con, bỏ đi tập rỗng còn $2^n - 1$.

■ "THEO VỊ TRÍ" KHÁC "KHÁC NHAU"

Định lí trên đếm theo vị trí. Nhiều bài toán lại hỏi số xâu con **khác nhau**. Với s = "aba", có 6 xâu con theo vị trí: "a", "b", "a", "ab", "ba", "aba"; nhưng chỉ có **5** xâu con khác nhau vì "a" xuất hiện hai lần. Đếm số xâu con khác nhau là một bài toán khó, sẽ được giải lần lượt bằng băm (Bài 11), mảng hậu tố (Bài 22) và ôtômát hậu tố (Bài 23).

Lê Thành Nam

3. Đọc và ghi xâu

C++ có hai cách đọc xâu thường dùng:

- cin >> s bỏ qua các khoảng trắng (dấu cách, xuống dòng, tab) ở đầu, rồi đọc tới khoảng trắng tiếp theo. Nó đọc **một từ**.
- getline(cin, s) đọc **cả một dòng**, kể cả dấu cách, tới kí tự xuống dòng (kí tự này bị lấy ra khỏi luồng nhưng không đưa vào s).

Trộn hai cách đọc này là nguồn gốc của một lỗi kinh điển. Giả sử dữ liệu vào có dòng đầu là số n, các dòng sau là họ tên:

‹› Lỗi "dòng rỗng" khi trộn cin >> và getline

```
int n;
cin >> n;           // đọc số 3, con trỏ dừng NGAY TRƯỚC kí tự xuống dòng
string name;
getline(cin, name);   // đọc tới kí tự xuống dòng đầu tiên: name = "" (rỗng!)
```



Hình 1.3. Sau cin >> n, kí tự xuống dòng vẫn còn trong luồng nhập

Có hai cách sửa: gọi cin.ignore() để bỏ đi kí tự xuống dòng còn sót, hoặc an toàn hơn là viết getline(cin >> ws, name): thao tác ws bỏ qua mọi khoảng trắng trước khi getline bắt đầu đọc.

■ KÍ TỰ '\r' Ở CUỐI DÒNG

Tệp tạo trên Windows kết thúc mỗi dòng bằng hai kí tự '\r' '\n'. getline chỉ bỏ '\n', nên s còn dính kí tự '\r' (mã 13) ở cuối. Hai xâu "abc" và "abc\r" in ra trông giống hệt nhau nhưng so sánh thì khác. Khi dùng getline, nên thêm dòng: if (!s.empty() && s.back() == '\r') s.pop_back();

Khi cần đọc và ghi rất nhiều dữ liệu (cỡ 10^6 kí tự trở lên), luôn đặt hai dòng ios_base::sync_with_stdio(false); cin.tie(nullptr); ở đầu hàm main, và in xuống dòng bằng '\n' thay cho endl (endl buộc đẩy bộ đệm ra màn hình, rất chậm khi gọi hàng trăm nghìn lần).

4. Các thao tác trên string và độ phức tạp

Kiểu string của C++ rất tiện, nhưng mỗi thao tác có một cái giá. Bảng sau là "bảng giá" mà em cần nhớ, với n là độ dài xâu s:

Thao tác	Chi phí	Ghi chú
s.size(), s[i], s.back()	O(1)	
s += c, s.push_back(c)	O(1) khấu hao	thỉnh thoảng phải xin chỗ mới
s += t	O(|t|)	chỉ sao chép t
s = s + c	**O(n)**	tạo xâu mới, sao chép cả s
s + t (tạo xâu mới)	O(n + |t|)	
s.substr(i, k)	O(k)	tạo một xâu mới độ dài k
s.insert(i, t), s.erase(i, k)	O(n)	dồn phần phía sau
s == t, s < t	O(min(n, |t|))	so sánh từng kí tự
s.find(t)	O(n·|t|) xấu nhất	thư viện không dùng KMP
reverse(s.begin(), s.end())	O(n)	
sort(s.begin(), s.end())	O(n log n)	

Dòng in đậm cần được giải thích kĩ.

Lê Thành Nam

■ DỰNG XÂU TỪNG KÍ TỰ

Dựng một xâu độ dài n bằng n lần `s = s + c` tốn tổng cộng n(n + 1)/2 lần sao chép kí tự, tức $\Theta(n^2)$. Dựng bằng n lần `s += c` chỉ tốn O(n).

■ CHỨNG MINH

Biểu thức `s + c` tạo ra một xâu mới có độ dài |s| + 1, nên phải sao chép toàn bộ s sang chỗ mới; sau đó phép gán mới chép kết quả vào s. Ở lần thứ i, s đã có i − 1 kí tự, nên tổng số kí tự bị sao chép ít nhất là 0 + 1 + ... + (n − 1) = n(n − 1)/2.

Ngược lại, `s += c` ghi c ngay vào cuối s nếu còn chỗ. Một `string` luôn xin trước một vùng nhớ có dung lượng (capacity) lớn hơn độ dài; khi đầy, nó xin vùng mới gấp đôi và chuyển các kí tự sang. Các lần chuyển xảy ra khi độ dài đạt 1, 2, 4, 8, ..., nên tổng số kí tự bị chuyển là 1 + 2 + 4 + ... + 2^k < 2n. Cộng với n lần ghi trực tiếp, tổng chi phí nhỏ hơn 3n.



Hình 1.4. Bên trái: mỗi lần s = s + c sao chép lại cả xâu. Bên phải: s += c chỉ sao chép khi dung lượng tăng gấp đôi

Lập luận "chi phí lớn thỉnh thoảng mới xảy ra, chia đều cho mọi thao tác thì nhỏ" được gọi là phân tích khấu hao. Ta sẽ gặp lại nó khi chứng minh thuật toán KMP (Bài 8) và thuật toán Manacher (Bài 14) chạy trong O(n).

■ NHỮNG VÒNG LẶP "VÔ TÌNH" O(n^{2})

- `res = res + c;` trong vòng lặp: mỗi lần O(n). Hãy viết `res += c;`.
- `s.erase(0, 1);` hoặc `s.insert(0, 1, c);` trong vòng lặp: mỗi lần dồn cả xâu, O(n).
- `for (int i = 0; i < strlen(a); i++)` với mảng kí tự kiểu C: `strlen` đếm lại độ dài sau mỗi vòng, O(n) mỗi lần. Hãy tính độ dài một lần trước vòng lặp.
- `t = s.substr(i, k);` trong vòng lặp lồng nhau: mỗi lần tạo một xâu mới độ dài k.
- Truyền xâu vào hàm theo giá trị `void f(string s)` mà hàm được gọi n lần: mỗi lần sao chép cả xâu. Hãy truyền tham chiếu `void f(const string &s)`.

Lê Thành Nam

■ GHI NHỚ

- Mỗi kí tự là một số. Chữ số, chữ hoa, chữ thường là ba khối mã liên tiếp: c − '0', c − 'a', c − 'A' + 'a'.
- Xâu con và tiền tố, hậu tố là đoạn **liên tiếp**; dãy con thì **không cần** liên tiếp. Có n(n + 1)/2 xâu con theo vị trí, $2^n - 1$ dãy con khác rỗng.
- cin >> s đọc một từ, getline đọc một dòng; sau cin >> n muốn getline thì dùng getline(cin >> ws, s).
- s += c là O(1) khấu hao; s = s + c là O(n). Cảnh giác với erase, insert, substr, find trong vòng lặp.

PHẦN BÀI TOÁN

■ BÀI TOÁN: LOCXAU

Cho một dòng văn bản s gồm các kí tự ASCII in được (mã từ 32 tới 126). Hãy tạo xâu mới gồm **các chữ cái và chữ số** của s, giữ nguyên thứ tự, trong đó mọi chữ cái hoa được đổi thành chữ thường.

Dữ liệu vào: Một dòng duy nhất là xâu s ($1 \le |s| \le 10^6$), có thể chứa dấu cách. Dữ liệu đảm bảo s có ít nhất một chữ cái hoặc chữ số.

Kết quả: Dòng thứ nhất là xâu mới. Dòng thứ hai gồm hai số: số chữ cái và số chữ số trong xâu mới.

Dữ liệu vào	Kết quả	Giải thích
Hello, World 2026!	helloworld2026 10 4	Bỏ dấu phẩy, dấu cách, dấu chấm than; "H", "W" thành "h", "w".
x = 10 + 25	x1025 1 4	Dữ liệu có dấu cách nên phải đọc bằng getline.

Lê Thanh Nam

5. Cách chậm: res = res + c

Cách làm tự nhiên nhất: đọc cả dòng bằng getline, duyệt từng kí tự, đổi chữ hoa thành chữ thường, rồi nối kí tự vào xâu kết quả.

▣ Nối bằng res = res + c, O(n^2)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s, res = "";
    getline(cin, s);
    int chu = 0, so = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        char c = s[i];
        if ('A' <= c && c <= 'Z') c = c - 'A' + 'a';
        if ('a' <= c && c <= 'z') { res = res + c; chu++; }  // chép cả res
```

Nối bằng res = res + c, O(n^2)

```
12        else if ('0' <= c && c <= '9') { res = res + c; so++; }
13    }
14    cout << res << '\n' << chu << ' ' << so << '\n';
15    return 0;
16 }
```

Chương trình đúng hoàn toàn về logic. Nó chỉ sai ở **một phép toán**: dòng 11 và dòng 12 tạo ra một xâu mới mỗi lần thêm một kí tự.

6. Cách nhanh: res += c

Chỉ cần đổi res = res + c thành res += c. Chương trình mẫu còn gọi res.reserve(s.size()) để xin trước đủ chỗ, nhờ vậy không lần tăng dung lượng nào xảy ra; dòng này không bắt buộc, vì theo định lí ở mục 4, += đã là O(1) khấu hao.

LOCXAU.CPP: Nối bằng res += c, O(n)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7     if (fopen("LOCXAU.INP", "r")) {
8        freopen("LOCXAU.INP", "r", stdin);
9        freopen("LOCXAU.OUT", "w", stdout);
10    }
11    string s, res;
12    getline(cin, s);                    // đọc cả dòng, kể cả dấu cách
13    if (!s.empty() && s.back() == '\r') s.pop_back();
14    res.reserve(s.size());              // xin trước đủ chỗ
15    int chu = 0, so = 0;
16    for (char c : s) {
17       if ('A' <= c && c <= 'Z') c = c - 'A' + 'a';   // hoa -> thường
18       if ('a' <= c && c <= 'z') { res += c; chu++; } // O(1) khấu hao
19       else if ('0' <= c && c <= '9') { res += c; so++; }
20    }
21    cout << res << '\n' << chu << ' ' << so << '\n';
22    return 0;
23 }
```

Lê Thành Nam

▪ ĐẾM SỐ PHÉP TÍNH

• res = res + c: $\Theta(n^2)$ lần sao chép kí tự. Với $|s| = 10^5$ mất **0,10 giây**; với $|s| = 2 \cdot 10^5$ mất **0,33 giây**; với $|s| = 10^6$ mất tới **10,8 giây**. Độ dài tăng 10 lần mà thời gian tăng hơn 100 lần: xâu càng dài thì càng không vừa bộ nhớ đệm của bộ xử lí, mỗi lần sao chép càng chậm.

• res += c: O(n). Với $|s| = 10^6$ chỉ mất **0,012 giây**, phần lớn là thời gian đọc dữ liệu.

■ BẪY THƯỜNG GẶP

• **Đọc bằng cin >> s:** chỉ đọc được từ đầu tiên "Hello,". Dữ liệu có dấu cách thì phải dùng getline.

• **In toupper/tolower mà không ép kiểu:** in ra mã số thay vì kí tự.

• **Quên kí tự '\r'** khi dữ liệu được tạo trên Windows: với bài này '\r' không phải chữ cái nên tự bị loại, nhưng ở những bài so sánh xâu thì nó gây sai.

• **Dùng mảng char cỡ nhỏ**, ví dụ char s[100000] cho xâu 10^6 kí tự: tràn mảng.

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	ĐẾM CHỮ	Mảng cnt[26], đọc cả dòng bằng getline, chú ý chữ hoa.
1.2	MẬT MÃ CAESAR	Dịch vòng c − 'a', lấy dư cho 26; cẩn thận k âm và k rất lớn.
1.3	ĐẦU CUỐI	Chèn vào đầu xâu là O(n): dùng deque hoặc hai xâu.

Tự kiểm tra nhanh:

1. Biểu thức '9' − '0' + 'a' cho kí tự nào?

(Đáp số: '9' − '0' = 9, 'a' + 9 là kí tự 'j'.)

2. Xâu "aaaa" có bao nhiêu xâu con theo vị trí, và bao nhiêu xâu con khác nhau?

(Đáp số: 4·5/2 = 10 xâu con theo vị trí, nhưng chỉ có 4 xâu con khác nhau: "a", "aa", "aaa", "aaaa".)

3. Đoạn lệnh string t = ""; for (int i = 0; i < n; i++) t = s[i] + t; có độ phức tạp bao nhiêu và làm gì?

(Đáp số: Nó đảo ngược xâu s, nhưng mỗi lần tạo xâu mới nên mất $\Theta(n^2)$. Nên dùng t = s; reverse(t.begin(), t.end());*, $O(n)$.)*

Lê Thành Nam

BÀI 2 Duyệt và biến đổi xâu

Học xong bài này, em sẽ:

✓ Thành thạo các **mẫu duyệt xâu**: duyệt từng kí tự, hai con trỏ từ hai đầu, gom nhóm các kí tự liên tiếp giống nhau.

✓ Kiểm tra xâu đối xứng, đảo xâu, nén loạt và giải nén trong O(n).

✓ **Tách từ chuẩn hoá** văn bản; chuyển đổi giữa xâu và số; so sánh 2 số viết dưới dạng xâu.

✓ Biết vì sao "sửa tại chỗ bằng erase" chậm, và viết lại bằng mẫu **con trỏ đọc – con trỏ ghi**.

■ KHỞI ĐỘNG

Một dòng văn bản dài 10^6 kí tự, các từ cách nhau bởi rất nhiều dấu cách. Cần chuẩn hoá: giữa hai từ chỉ còn một dấu cách. Cách nghĩ đầu tiên: "gặp hai dấu cách liền nhau thì xoá một cái", bằng s.erase(i, 1). Chương trình ấy đúng, nhưng chạy mất **16,0 giây**. Viết lại bằng cách **dựng một xâu mới** thì chỉ còn 0,012 giây. Bài học này tổng kết những mẫu duyệt xâu cơ bản nhất, và rèn thói quen luôn tự hỏi: "mỗi kí tự được chạm tới bao nhiêu lần?"

PHẦN LÝ THUYẾT

1. Ba cách duyệt một xâu

Mọi thuật toán trên xâu đều bắt đầu bằng việc duyệt các kí tự. C++ cho ba cách viết:

‹› Ba cách duyệt

```
for (int i = 0; i < (int)s.size(); i++) { ... s[i] ... }  // cần chỉ số i
for (char c : s) { ... c ... }        // chỉ đọc: c là bản sao của từng kí tự
for (char &c : s) { c = tolower(c); } // muốn sửa: c là tham chiếu
```

Ở cách thứ nhất, nhớ ép s.size() về int. Hàm size() trả về số không dấu; với xâu rỗng, s.size() − 1 **không** bằng −1 mà bằng một số rất lớn (khoảng $1{,}8 \cdot 10^{19}$), nên vòng lặp for (int i = 0; i < s.size() − 1; i++) sẽ chạy loạn.

2. Hai con trỏ từ hai đầu: đối xứng và đảo xâu

■ XÂU ĐỐI XỨNG

Xâu s độ dài n là **xâu đối xứng** (palindrome) nếu đọc xuôi và đọc ngược như nhau, tức s[i] = s[n − 1 − i] với mọi $0 \le i < n$. Ví dụ: "racecar", "abba", "a"; xâu rỗng cũng được coi là đối xứng.

Để kiểm tra, đặt con trỏ i ở đầu, j ở cuối, so sánh s[i] với s[j] rồi cùng tiến vào giữa. Chỉ cần so tới khi $i \ge j$: mỗi cặp được so đúng một lần, và kí tự ở tâm (khi n lẻ) không cần so với ai.